

100 SCENARIO-BASED AI/ML Engineer Interview Questions

MNC-style situational rounds — Google • Microsoft • NASA • Tesla • Nvidia • Amazon

Each question is a real-world scenario, not a definition. The goal is to test whether a candidate can reason about constraints (data, latency, cost, safety) and pick the right model/technique — the way a real interviewer at a top company actually evaluates you.

How to use this document

1. Go section by section. Read ONLY the scenario + hint, and speak your answer out loud (or write it) before checking the answer key.
2. Don't peek at the Answer Key section at the back until you've genuinely attempted the question — that's the whole point of a mock interview.
3. The 'ideal direction' in the answer key is not the only correct answer — in a real interview, reasoning and the right questions asked matter more than matching this exactly.

Prepared for: Ayan's Deep Learning Journey
github.com/aryantyagi2211/DeepLearning_Journey

CONTENTS

Section	Topic	Q#
Section A	MODEL SELECTION & ML FUNDAMENTALS	Q1-Q12
Section B	COMPUTER VISION / CNN SCENARIOS	Q13-Q28
Section C	SEQUENCE MODELS / TIME SERIES SCENARIOS	Q29-Q38
Section D	NLP SCENARIOS	Q39-Q52
Section E	TRANSFORMER / ATTENTION SCENARIOS	Q53-Q62
Section F	LLM / GENERATIVE AI SCENARIOS	Q63-Q82
Section G	SYSTEM DESIGN, MLOPS & PRODUCTION SCENARIOS	Q83-Q100
Answer Key	Ideal-direction answers for all 100 questions	back of document

SECTION A | MODEL SELECTION & ML FUNDAMENTALS

These test whether the candidate reaches for a neural network by default, or actually reasons about the data and constraints first — a huge red flag if they always say 'deep learning' without asking questions.

Q1

You're at Google working on ad click-through-rate prediction. The data is structured/tabular (user age, device, time of day, historical CTR, ~200 columns), with hundreds of millions of rows.

■ *Think about: does this problem actually need a neural network?*

Q2

At Microsoft, a team has only 300 labeled examples for a customer-churn classification task, but they insist on using a 10-layer deep neural network because 'deep learning is the future.'

■ *Think about: what happens to a huge model with very little data?*

Q3

NASA has satellite telemetry with millions of noisy sensor readings, and needs a system that outputs a well-calibrated probability of an anomaly, not just a yes/no label — because downstream mission-control decisions depend on the probability being trustworthy.

■ *Think about: what does 'well-calibrated probability' demand beyond raw accuracy?*

Q4

Tesla needs an on-vehicle model that must make a decision in under 10 milliseconds on embedded hardware with no cloud connection during highway driving.

■ *Think about: what trumps raw accuracy here?*

Q5

At Nvidia, a training job is using only 30% GPU utilization even though the model is large and the GPUs are powerful.

■ *Think about: is this a compute problem or something else?*

Q6

Amazon's recommendation team needs to recommend products to brand-new users who have zero purchase history (the 'cold start' problem).

■ *Think about: what can you use when there's no user history at all?*

Q7

A candidate shows you a training curve: training loss keeps dropping smoothly, but validation loss starts rising after epoch 15.

■ *Think about: what is this classic pattern called, and what's the fix?*

Q8

A fraud-detection model performed great in testing but its precision has been silently dropping in production for three months.

■ *Think about: what changes over time that testing doesn't capture?*

Q9

A bank at Google Cloud wants a model to approve/reject loans, but regulators require the bank to explain every rejection to the applicant.

■ *Think about: does the most accurate model win here?*

Q10

Microsoft's fraud team has a dataset where only 0.2% of transactions are fraudulent; a model trained naively gets 99.8% accuracy by predicting 'not fraud' every time.

■ *Think about: why is accuracy a misleading metric here?*

Q11

NASA has only 40 labeled examples of a rare satellite-component failure, but thousands of hours of normal telemetry.

■ *Think about: how do you learn from extremely rare labeled events?*

Q12

Tesla needs to deploy a vision model on an embedded chip with strict memory and power limits, but the best-performing model in testing is a huge network.

■ *Think about: how do you shrink a model without starting over?*

SECTION B | COMPUTER VISION / CNN SCENARIOS

Tests whether the candidate can match the right vision architecture and technique to real constraints (speed, data size, image type) instead of reciting CNN theory in a vacuum.

Q13 Tesla needs to detect pedestrians, cars, and traffic signs from a camera feed in real time while driving at highway speed, with a strict end-to-end latency budget.

■ Think about: two-stage vs single-stage object detectors.

Q14 NASA has only 500 labeled satellite images to classify terrain types, but access to large pretrained vision models.

■ Think about: from-scratch training vs reusing existing knowledge.

Q15 An Nvidia healthcare partner needs pixel-level segmentation of tumors in MRI scans, not just a bounding box or class label.

■ Think about: classification vs detection vs segmentation — which one actually answers the question?

Q16 Google wants to power 'search by image' — given a photo, find visually similar photos in a billion-image database, in near real time.

■ Think about: how do you compare millions of images fast?

Q17 A Microsoft manufacturing client needs to detect microscopic defects on circuit boards. They have only 800 defect images, but a missed defect is very costly, so false negatives must be minimized.

■ Think about: what does 'minimize false negatives' change about model choice/threshold, beyond just picking a CNN?

Q18 Tesla's vision team notices the model reliably detects nearby cars but frequently misses small, distant vehicles.

■ Think about: what part of the detector's design affects performance on small objects?

Q19 A team's 50-layer CNN has training loss that barely moves after a few epochs — it looks like the network just isn't learning.

■ Think about: what's a known failure mode of very deep plain CNNs?

Q20

Nvidia is building a 1:N face-matching system for access control across thousands of employees, where new employees are added constantly.

■ *Think about: standard softmax classification vs a different training objective for this problem.*

Q21

NASA has old, noisy scanned satellite images that need to be classified but the noise is hurting model accuracy.

■ *Think about: fix it at the model level or the data level?*

Q22

Google wants to automatically place bounding boxes around products in user-uploaded shopping images for an ad platform.

■ *Think about: classification alone isn't enough here — what's needed?*

Q23

A medical imaging classifier at a hospital partner has too many false positives, causing alert fatigue among doctors.

■ *Think about: how do you trade off precision and recall after the model is already trained?*

Q24

Tesla combines camera images and Lidar point clouds for object detection — the two sensors capture very different types of data.

■ *Think about: how do you combine fundamentally different data modalities in one model?*

Q25

A team training a CNN on very high-resolution (8K) images keeps hitting GPU out-of-memory errors.

■ *Think about: what are your options besides just buying a bigger GPU?*

Q26

A Microsoft warehouse-robotics client's detector is drawing 4-5 overlapping boxes around the same single box on a shelf.

■ *Think about: what post-processing step handles duplicate detections?*

Q27

A CNN classifies product images well when they're centered and upright, but accuracy drops sharply on the same products photographed at an angle or partially rotated.

■ *Think about: what training-time technique builds invariance to these transformations?*

Q28

Nvidia needs to run video classification on an edge device with very limited GPU memory and power budget, processing a live camera stream.

■ *Think about: which architecture family and optimization techniques target exactly this constraint?*

SECTION C | SEQUENCE MODELS / TIME SERIES SCENARIOS

Tests understanding of when RNN/LSTM/GRU genuinely fit a problem versus when a simpler statistical model or a Transformer is the smarter choice.

Q29 Tesla wants to predict battery degradation over the next 6 months using multi-year sequences of daily charge/discharge sensor readings per vehicle.

■ Think about: what type of model naturally handles ordered, evolving sensor sequences?

Q30 NASA needs to predict a spacecraft's future orbital position sequence given its recent trajectory readings.

■ Think about: single-value prediction vs predicting a whole future sequence.

Q31 Nvidia wants to forecast GPU cluster workload for the next month to plan capacity, using years of historical daily usage data with clear weekly/seasonal patterns.

■ Think about: do you need a deep sequence model at all, or would something simpler and more explainable work just as well?

Q32 A candidate's RNN training loss is spiking wildly, occasionally jumping to NaN, mid-training.

■ Think about: this is a very specific, well-known RNN training failure and its standard fix.

Q33 Microsoft needs to flag anomalies in a continuous stream of server logs in near real time, where the definition of 'normal' behavior can drift slowly over weeks.

■ Think about: does this need a heavy sequence model, and how do you handle a slowly changing 'normal'?

Q34 A team is trying to process sequences of 10,000+ timesteps with an LSTM and finds training extremely slow with the model still failing to capture early-sequence information by the end.

■ Think about: is LSTM fundamentally the wrong architecture at this sequence length?

Q35 Tesla needs a sequential model running on limited in-car compute where inference speed matters more than squeezing out the last bit of accuracy.

■ Think about: LSTM vs a lighter alternative with fewer gates.

Q36

A translation feature needs to convert variable-length input sentences into variable-length output sentences in a different language.

■ *Think about: what classic architecture pattern handles variable-length-in, variable-length-out?*

Q37

NASA's telemetry readings arrive at irregular time intervals (not a fixed timestep), which breaks the usual assumption behind standard RNN/LSTM inputs.

■ *Think about: what has to change about how the model is fed data when timesteps aren't evenly spaced?*

Q38

A junior teammate wants to throw an LSTM at raw daily stock prices to predict tomorrow's price, expecting strong results.

■ *Think about: is the core problem here about the model architecture at all?*

SECTION D | NLP SCENARIOS

Tests whether the candidate picks text techniques based on data volume, latency, and the actual linguistic problem — not just 'use BERT for everything.'

Q39 Google wants search queries to match semantically relevant documents, even when they don't share exact keywords (e.g. 'cheap flights' should match 'affordable airfare').

■ Think about: keyword matching vs meaning matching.

Q40 Microsoft's support team wants to auto-route incoming tickets into 20 categories. They have 500,000 historical labeled tickets.

■ Think about: with this much labeled data, is a huge pretrained LLM actually necessary?

Q41 An e-commerce platform needs sentiment analysis on millions of reviews daily at low cost, where a marginal accuracy gain isn't worth 10x the compute cost.

■ Think about: what's the right tradeoff between accuracy and serving cost here?

Q42 NASA needs to pull out mission names, dates, and instrument names from thousands of unstructured scientific reports.

■ Think about: what NLP task is specifically designed to extract this kind of information?

Q43 A voice assistant needs to figure out what a user wants ('book a flight', 'check weather', 'set an alarm') from short, sometimes ambiguous spoken queries.

■ Think about: what's this NLP task called, and does it typically need a huge model?

Q44 Google's search bar needs to autocomplete and correct typos in real time as the user types, with sub-50ms latency.

■ Think about: this needs to be fast — does that rule out certain model types?

Q45 A multinational company needs one support chatbot that works across English, Spanish, Hindi, and Mandarin without maintaining 4 separate models.

■ Think about: single-language models vs one shared representation.

Q46

Microsoft needs to automatically flag documents containing personal data (names, SSNs, emails) before they're shared externally, and false negatives (missed PII) are unacceptable.

■ *Think about: pure ML model vs a hybrid approach for a near-zero-false-negative requirement.*

Q47

A news aggregator wants to generate short summaries of long articles for a mobile app, prioritizing summaries that read naturally rather than just copying sentences.

■ *Think about: extractive vs abstractive summarization — which fits 'reads naturally'?*

Q48

A candidate's sentiment model consistently misclassifies sentences using the word 'sick' (as in 'sick beat' = positive slang vs 'I feel sick' = negative) because it uses static word vectors.

■ *Think about: what type of embedding is needed to resolve a word whose meaning depends on context?*

Q49

A Q&A forum at Google wants to detect when a newly posted question is a duplicate of an existing one, phrased differently.

■ *Think about: exact text match vs meaning-based comparison.*

Q50

A company has 5 million unlabeled support emails and only 2,000 labeled ones for a classification task.

■ *Think about: how do you make the huge unlabeled pool useful?*

Q51

A live customer-support widget needs to classify message urgency instantly as the user types, on a modest server budget.

■ *Think about: strict real-time constraint plus budget constraint — what does that rule out?*

Q52

NASA needs to pull structured fields (date, location, instrument reading, anomaly type) out of free-text mission logs written inconsistently by different engineers over decades.

■ *Think about: what pipeline design handles inconsistent, free-form text into structured fields?*

SECTION E | TRANSFORMER / ATTENTION SCENARIOS

Tests understanding of **why** attention/Transformers exist and when their specific properties (parallelism, long-range context, bidirectionality) actually matter.

Q53

A legal-document analysis tool needs to connect a pronoun on page 40 to the entity it refers to on page 2 of a long contract.

■ *Think about: why would an RNN struggle here that a Transformer wouldn't?*

Q54

Google's search re-ranking step needs to deeply compare a query and a candidate document together, jointly, to score relevance — not just compare their separate embeddings.

■ *Think about: a bi-encoder (separate embeddings) vs a cross-encoder (joint encoding) — which fits 'jointly compare'?*

Q55

A research team's RNN-based model takes 3 weeks to train on their GPU cluster and they want to cut that down significantly without changing the underlying task.

■ *Think about: what fundamental bottleneck of RNNs does a Transformer remove?*

Q56

Microsoft needs to classify 50,000-token legal documents, but a standard Transformer's self-attention cost grows quadratically with sequence length, making this too expensive.

■ *Think about: what are the practical options for handling sequences far longer than a standard Transformer's context window?*

Q57

A company wants one model that can do sentiment analysis, topic classification, and named entity recognition, without training and maintaining three completely separate models.

■ *Think about: shared representation with task-specific heads.*

Q58

Nvidia needs to deploy a Transformer model for inference on an edge GPU with a strict memory ceiling, where the full model doesn't fit.

■ *Think about: what techniques specifically shrink Transformer inference footprint?*

Q59

A creative-tools startup wants users to type a text prompt and get a relevant image caption/description generated based on both the uploaded image and the text prompt together.

■ *Think about: single-modality model vs something that jointly reasons over image and text.*

Q60

An interviewer asks: 'Why would you use BERT instead of GPT to classify whether a customer review is positive or negative?'

■ *Think about: does generation ability matter for this task, and does bidirectional context help?*

Q61

The same interviewer then asks: 'Why would you use GPT instead of BERT to build a chatbot that writes free-form responses?'

■ *Think about: which architecture's core training objective naturally supports generating new text token by token?*

Q62

A candidate finds that their trained Transformer's attention weights are almost uniformly spread across all tokens instead of focusing on relevant ones, and outputs are poor.

■ *Think about: what could cause attention to fail to specialize/focus at all?*

SECTION F | LLM / GENERATIVE AI SCENARIOS

This is the highest-weight section in 2026 interviews. Tests whether the candidate defaults to 'just fine-tune an LLM' or actually reasons about RAG, cost, latency, safety, and grounding.

Q63 Google wants an internal assistant that answers employee questions using proprietary, constantly-updated internal documentation.

■ *Think about: fine-tuning vs a retrieval-based approach for knowledge that changes often.*

Q64 Microsoft wants LLM responses to consistently reflect company tone and specific product knowledge, but retraining a full model for every product update is too expensive.

■ *Think about: is this a tone problem, a knowledge problem, or both — and what techniques address each cheaply?*

Q65 Nvidia's API team needs to serve an LLM chatbot to millions of users with a strict per-query cost and latency budget.

■ *Think about: what levers reduce cost/latency at inference without retraining the model?*

Q66 NASA wants a domain-specific assistant for space-mission engineering documents, but has very little labeled instruction-following data in that domain.

■ *Think about: fine-tuning needs labeled data — what's the alternative when that data is scarce?*

Q67 A customer-facing product's LLM chatbot occasionally states incorrect facts confidently, damaging user trust.

■ *Think about: this is the classic 'hallucination' problem — what are the standard mitigations?*

Q68 A legal-assistant LLM product needs to maintain coherent context across a conversation spanning hundreds of turns, exceeding the model's trained context window.

■ *Think about: what are the engineering options when a conversation outgrows the context window?*

Q69 Tesla wants an in-car voice assistant that must keep working even with no internet connectivity (tunnels, remote areas).

■ *Think about: cloud LLM API vs something that must run fully offline.*

Q70

A company's chatbot is popular but GPU serving costs are becoming unsustainable at current scale.

■ *Think about: multiple levers to cut serving cost — not just 'buy fewer GPUs'.*

Q71

A general-purpose LLM assistant needs to refuse harmful requests (e.g. building weapons) while staying maximally helpful for legitimate technical questions in adjacent domains (e.g. chemistry homework).

■ *Think about: what's the underlying tension being designed for here, and what techniques address it?*

Q72

A healthcare company wants to fine-tune an LLM on patient records but is worried about privacy leakage from the trained model memorizing sensitive data.

■ *Think about: fine-tuning risk with sensitive data vs an alternative that avoids baking data into model weights.*

Q73

Google wants LLM-generated answers to always show which source document each claim came from, for auditability.

■ *Think about: what architecture naturally supports traceable, sourced answers?*

Q74

An LLM-powered hiring-assistant tool is found to rank resumes differently based on names that suggest different demographic groups, even when qualifications are identical.

■ *Think about: what's the process for identifying and fixing this, beyond just 'retrain the model'?*

Q75

A startup with a small budget wants to add an LLM feature quickly, and the team is debating fine-tuning a model from day one.

■ *Think about: what's the cheapest, fastest thing to try first, and when do you escalate?*

Q76

Nvidia's LLM API needs higher token-generation throughput to serve more concurrent users on the same hardware.

■ *Think about: inference-time techniques that increase throughput without changing model quality.*

Q77

An LLM tutoring app gives wrong answers to multi-step math word problems even though it clearly 'knows' the individual facts involved.

■ *Think about: what prompting/architecture technique specifically helps multi-step reasoning?*

Q78

A product team wants their LLM assistant to actually check a live calendar and book a meeting, not just describe how the user could do it themselves.

■ *Think about: what capability turns an LLM from a text generator into something that takes real actions?*

Q79

Microsoft's multi-tenant enterprise LLM platform must guarantee that Customer A's fine-tuned model or conversation data can never leak into Customer B's responses.

■ *Think about: is this primarily a modeling problem or an architecture/infrastructure problem?*

Q80

Before launching a new LLM feature, a team needs to decide whether the new model version is actually better than the old one.

■ *Think about: what does a proper LLM evaluation setup look like, beyond 'it feels better'?*

Q81

A legal-tech LLM performs noticeably worse on specialized legal terminology than on everyday language.

■ *Think about: is this a knowledge gap or a tokenization gap, and how do you tell?*

Q82

A small startup wants GPT-level capability for their product but can't afford to pretrain a large model from scratch.

■ *Think about: what's the realistic, standard path here?*

SECTION G | SYSTEM DESIGN, MLOPS & PRODUCTION SCENARIOS

These separate candidates who've only trained models in notebooks from those who understand what it takes to run ML safely and reliably at MNC scale.

Q83

Tesla's autonomous driving stack must never silently fail — if the model's confidence drops or something looks wrong, the car needs a safe fallback.

■ *Think about: what's the design principle for safety-critical ML systems, beyond making the model as accurate as possible?*

Q84

Nvidia's multi-day distributed training job on hundreds of GPUs keeps crashing randomly due to hardware failures, losing all progress each time.

■ *Think about: what standard practice prevents losing days of training progress?*

Q85

Google notices a production recommendation model's engagement metrics slowly declining over several months, with no code changes.

■ *Think about: what's happening to the data itself over time, and how do you catch it?*

Q86

NASA needs to deploy an ML model onboard a spacecraft where, once launched, software updates are extremely difficult or impossible.

■ *Think about: how does the total lack of easy updates change your entire development approach?*

Q87

Microsoft wants to roll out a new model version to its millions of Office users without risking a bad experience if the new model underperforms.

■ *Think about: how do you deploy a new model safely at massive scale?*

Q88

A team can't reproduce their own model's results — the same code and data gives different metrics each run.

■ *Think about: what sources of nondeterminism cause this, and how do you eliminate them?*

Q89

Amazon needs product recommendations served in under 100 milliseconds at massive scale across millions of concurrent users.

■ *Think about: what makes this a systems problem as much as a modeling problem?*

Q90

A team building a fraud-detection model is unsure whether to optimize for precision, recall, F1, or AUC, and picks one somewhat arbitrarily.

■ *Think about: what should actually drive this decision?*

Q91

Tesla needs to validate a new autonomous-driving model update before pushing it over-the-air to millions of cars.

■ *Think about: what layers of validation happen before millions of real cars get this update?*

Q92

A company's training pipeline silently started producing a broken/garbage model a week ago, and nobody noticed until customers complained.

■ *Think about: what should have caught this automatically?*

Q93

Google has a training dataset too large to fit on a single machine's memory or disk.

■ *Think about: what training strategy handles data (or model) larger than one machine?*

Q94

Nvidia is training a model so large it doesn't fit in a single GPU's memory, and the team is deciding between data parallelism and model parallelism.

■ *Think about: what determines which parallelism strategy — or combination — is actually needed?*

Q95

A startup is debating cloud GPUs vs buying on-premise GPU hardware for their ML training needs.

■ *Think about: what factors actually determine which is cheaper/better, beyond sticker price?*

Q96

Microsoft receives a GDPR-style request from a user demanding their personal data — and any model trained on it — be deleted.

■ *Think about: deleting the raw data is easy; what's the harder part?*

Q97

A model performs well in development testing but fails badly once deployed, and the team suspects the input data looks different in production than during training.

■ *Think about: what's this classic issue called, and how do you prevent it?*

Q98

NASA needs a model's prediction to come with a trustworthy confidence/uncertainty estimate before mission control acts on it, since a wrong high-confidence prediction could be catastrophic.

■ *Think about: what techniques give genuine uncertainty estimates, not just a raw softmax score?*

Q99

A candidate is asked in an interview: 'Your model gets 99% training accuracy but only 50% test accuracy. Walk me through how you'd debug this.'

■ *Think about: what's your structured diagnostic process, not just the one-word answer 'overfitting'.*

Q100

CAPSTONE: You're asked to design an end-to-end ML system from scratch for a brand-new problem at your company (the interviewer deliberately leaves the problem statement open-ended, e.g. 'design a system to detect equipment failure before it happens'). You have 15 minutes on a whiteboard.

■ *Think about: what's your structured framework for approaching ANY open-ended ML system design question, in order?*

ANSWER KEY

Ideal-direction answers for all 100 scenarios. These describe the reasoning and approach a strong candidate would give — not rigid word-for-word scripts.

SECTION A | MODEL SELECTION & ML FUNDAMENTALS

Q1.

Prefer Gradient Boosted Trees (XGBoost/LightGBM) for structured tabular data — they usually beat deep nets on tabular data, train faster, and are more interpretable. A deep net (wide & deep) is justified only if you need to fuse tabular with embeddings/text/image.

Q2.

With only 300 samples, a deep net will overfit badly. Push back — recommend a simpler model (logistic regression, small tree ensemble) or heavy regularization/transfer learning, and get more labeled data if possible.

Q3.

Optimize for and evaluate calibration explicitly (Brier score / calibration curves), not just accuracy. Use techniques like temperature scaling, Platt scaling, or Bayesian/ensemble approaches, and choose a loss function (e.g. cross-entropy) that yields meaningful probabilities.

Q4.

Latency and determinism trump peak accuracy — choose a lightweight architecture (small CNN/MobileNet-class), quantize/prune it, and benchmark worst-case latency on the actual target hardware, not just average-case.

Q5.

This is very likely a data-loading / I/O bottleneck (CPU preprocessing, disk reads, small batch size) rather than a model-compute problem — profile the pipeline before assuming you need a bigger GPU or different model.

Q6.

Pure collaborative filtering fails here — use a hybrid approach: content-based features (item metadata, demographic priors) combined with collaborative filtering once enough interaction data accumulates.

Q7.

Classic overfitting. Fixes: early stopping around the point validation loss turns up, add dropout/regularization/weight decay, get more data or augment, or reduce model capacity.

Q8.

This points to data/concept drift — the distribution of fraud patterns changed after deployment. Solution: continuous monitoring of input distributions and live metrics, with a scheduled or trigger-based retraining pipeline.

Q9.

Interpretability is a hard constraint, not a nice-to-have. Prefer inherently interpretable models (logistic regression, decision trees) or pair a complex model with explainability tools (SHAP/LIME) — and be ready to justify the tradeoff of losing a little accuracy.

Q10.

Accuracy is meaningless under extreme class imbalance. Use precision/recall/F1/PR-AUC instead, and address the imbalance with class weighting, oversampling (SMOTE) or undersampling, and threshold tuning based on business cost of false negatives vs false positives.

Q11.

Treat this as anomaly detection (model 'normal' behavior, flag deviations) rather than a standard supervised classifier, and/or use transfer learning or few-shot techniques leveraging the abundant normal data.

Q12.

Apply model compression: quantization (lower precision weights), pruning (remove low-impact weights/channels), and/or knowledge distillation (train a small 'student' model to mimic the large 'teacher').

SECTION B | COMPUTER VISION / CNN SCENARIOS

Q13.

Choose a single-stage detector (YOLO-family or SSD) over two-stage detectors (Faster R-CNN) — single-stage trades a little accuracy for the real-time speed that safety-critical driving requires.

Q14.

Use transfer learning: take a backbone pretrained on ImageNet (or a satellite-domain pretrained model if available) and fine-tune the last layers on the 500 labeled images — training from scratch on 500 images will badly overfit.

Q15.

This calls for a segmentation architecture like U-Net (or a modern variant) which outputs a pixel-wise mask, not a classification or object-detection network.

Q16.

Use a CNN (or vision transformer) to produce fixed-length embeddings for every image, then do approximate nearest-neighbor search (e.g. FAISS/HNSW) in embedding space — exact pairwise comparison doesn't scale to a billion images.

Q17.

Use transfer learning given the small dataset, but also explicitly bias the decision threshold toward higher recall (accepting more false positives) and consider a cost-sensitive loss function, since a missed defect is far more expensive than a false alarm.

Q18.

This is a multi-scale detection problem — check and tune anchor box sizes/scales, use a feature pyramid to combine features from multiple resolutions, and consider training with more small-object examples.

Q19.

Classic vanishing-gradient symptom in very deep plain networks. Fix with residual/skip connections (ResNet-style), batch normalization, and proper weight initialization (He init for ReLU).

Q20.

Standard classification doesn't scale (you'd need to retrain every time someone new joins). Use metric/embedding learning (triplet loss or contrastive loss) so the model learns a similarity space, and new people can be added just by storing their embedding.

Q21.

Handle noise at the data/preprocessing level first: denoising filters, contrast normalization, and aggressive/robust data augmentation during training so the model becomes tolerant to similar noise patterns.

Q22.

This is an object detection task (needs both 'what' and 'where'), not plain classification — use a detector like Faster R-CNN/YOLO depending on the latency requirement of the ad pipeline.

Q23.

Tune the classification decision threshold to favor precision over recall (fewer, more confident positive alerts), and/or use a cost-sensitive loss during training that penalizes false positives more heavily, guided by clinician input on acceptable tradeoffs.

Q24.

This needs a sensor-fusion / multi-modal architecture that processes each modality with an appropriate encoder (CNN for camera, point-cloud network for Lidar) and fuses the resulting features, rather than forcing both into one identical pipeline.

Q25.

Options: train on image patches/crops instead of full resolution, downsample images if fine detail isn't essential, reduce batch size, or use gradient checkpointing/mixed-precision training to cut memory use.

Q26.

This is a Non-Max Suppression (NMS) issue — the IoU threshold used to merge/remove overlapping boxes needs tuning; too loose a threshold leaves duplicates, too tight can wrongly remove genuinely separate nearby objects.

Q27.

The model hasn't learned rotation/scale invariance from its training data — add data augmentation (rotation, scaling, cropping, flipping) during training so the model sees these variations and generalizes to them.

Q28.

Use a lightweight architecture designed for edge devices (MobileNet/EfficientNet-lite class) combined with quantization, and consider processing at a reduced frame rate/resolution to stay within the power and memory budget.

SECTION C | SEQUENCE MODELS / TIME SERIES SCENARIOS

Q29.

An LSTM (or GRU) is a natural fit for this sequential, temporally-dependent sensor data — it can capture long-term degradation trends across the sequence, unlike a model that treats each day independently.

Q30.

This needs a sequence-to-sequence model (encoder-decoder LSTM/GRU, or Transformer-based) since the output is itself a sequence of future positions, not a single next value.

Q31.

For clearly seasonal, moderately-sized time series, classical models (ARIMA/Prophet) or gradient-boosted models with lag features are often as good as or better than an LSTM, train much faster, and are easier to explain to stakeholders — deep learning isn't automatically the right answer.

Q32.

This is exploding gradients — apply gradient clipping (cap the gradient norm during backprop), and consider a lower learning rate or switching to LSTM/GRU which are more stable than vanilla RNN.

Q33.

A lightweight streaming/online approach (statistical control charts, or a small online-updated model) is often more practical than a heavy offline-trained LSTM here, specifically because the model needs to keep adapting to a drifting definition of normal in real time.

Q34.

At this sequence length, LSTMs struggle with both training speed (still sequential) and long-range memory. A Transformer-based architecture (which processes the sequence in parallel and models long-range dependencies via attention) is the better fit.

Q35.

Prefer GRU over LSTM — GRU has fewer parameters and gates (no separate cell state), so it's faster at inference with usually comparable accuracy to LSTM on many tasks.

Q36.

This is the textbook Seq2Seq (encoder-decoder) use case, ideally with an attention mechanism (or a full Transformer) so the decoder can reference relevant parts of the input regardless of output length.

Q37.

Standard RNNs assume regular timesteps. You need to explicitly encode the time gaps (e.g. feed delta-time as a feature, use time-aware RNN variants like Time-LSTM, or resample/interpolate to a regular grid) before or during modeling.

Q38.

Push back on the framing — financial price series are dominated by near-random-walk noise; architecture choice (LSTM vs anything else) is secondary to whether there's genuine predictive signal in the features at all. Better engineered features and realistic expectations matter more than swapping model types.

SECTION D | NLP SCENARIOS

Q39.

Keyword-based search (TF-IDF/BM25) misses this — use dense semantic embeddings (from a Transformer-based sentence encoder) so queries and documents are compared by meaning, not exact words.

Q40.

With half a million labeled examples, a simpler and cheaper approach (TF-IDF + classical classifier, or fine-tuning a small pretrained encoder like DistilBERT) will likely perform very well and be far cheaper to serve than a large LLM — match model size to the problem, not just what's trendy.

Q41.

Choose a lightweight/distilled model (or classical ML on TF-IDF features) rather than a large Transformer — at this scale, cost-per-inference dominates, and the accuracy gap for sentiment analysis is often small.

Q42.

This is a Named Entity Recognition (NER) task — use (or fine-tune) an NER model, potentially combined with domain-specific entity lists/gazetteers for space-mission terminology not seen in general-purpose NER training data.

Q43.

This is intent classification — usually solvable well with a moderately-sized fine-tuned classifier over short text; a huge LLM is often overkill for a fixed, well-defined set of intents unless the intent set is open-ended or constantly expanding.

Q44.

This needs a lightweight, low-latency language model (small n-gram or compact neural LM) rather than a large Transformer — the strict latency budget rules out anything heavy, even if it would be slightly more accurate.

Q45.

Use a multilingual pretrained model (multilingual BERT/XLM-R style, or a multilingual LLM) trained on shared cross-lingual embeddings, rather than maintaining separate monolingual pipelines.

Q46.

Use a hybrid: NER model for names/entities plus deterministic regex/pattern rules for structured PII (SSNs, emails, phone numbers) — rules give near-perfect recall on structured patterns that a purely learned model might miss.

Q47.

Choose abstractive summarization (generates new phrasing, e.g. via an encoder-decoder or LLM) over extractive summarization (which just selects/copies existing sentences and can read choppy) since natural readability is the stated priority.

Q48.

Static embeddings (Word2Vec/GloVe) give 'sick' one fixed vector regardless of context — switch to contextual embeddings (BERT-style) which generate a different representation for the same word depending on surrounding context.

Q49.

Generate sentence embeddings for questions and compare via cosine similarity/nearest-neighbor search — duplicate detection is fundamentally a semantic similarity problem, not exact string matching.

Q50.

Use the unlabeled data for self-supervised pretraining (or start from an already-pretrained language model) and then fine-tune on the 2,000 labeled examples — this leverages the unlabeled data indirectly instead of wasting it.

Q51.

Rules out large Transformer models for this specific classification step — use a lightweight classifier (small fine-tuned encoder or classical ML on embeddings) that meets the latency and cost budget, reserving heavier models for non-real-time tasks.

Q52.

Design an information-extraction pipeline: NER/entity extraction plus relation extraction (or a fine-tuned LLM prompted to extract structured JSON), likely needing a normalization step afterward since terminology varies across decades of logs.

SECTION E | TRANSFORMER / ATTENTION SCENARIOS

Q53.

This needs long-range dependency modeling — self-attention can directly connect distant tokens in one step, while an RNN would have to carry that information through dozens of sequential hidden-state updates, which tends to fade over that distance.

Q54.

Use a cross-encoder (query and document fed together into the same Transformer, using cross-attention) for high-precision re-ranking of a shortlist — bi-encoders are faster for the initial large-scale retrieval, but cross-encoders give higher-quality relevance scores for final ranking.

Q55.

Switch to a Transformer-based architecture — RNNs process tokens sequentially (step t needs step $t-1$), which prevents full parallelization; Transformers process the whole sequence in parallel via self-attention, dramatically speeding up training on GPUs.

Q56.

Options: chunk the document and aggregate results, use a long-context Transformer variant with efficient/sparse attention, or use a hierarchical approach (summarize sections, then classify the summary) — plain full self-attention on 50k tokens is computationally impractical.

Q57.

Use a shared Transformer encoder (pretrained, possibly fine-tuned) with separate lightweight task-specific output heads for each task — this is standard multi-task learning, cheaper to maintain than three fully separate models.

Q58.

Apply knowledge distillation (train a smaller Transformer to mimic the large one), quantization (reduce weight precision), and attention-efficient variants (e.g. GQA) to reduce KV-cache memory — a combination is usually needed to hit a strict memory ceiling.

Q59.

This needs a multimodal architecture — an image encoder (CNN/ViT) feeding into a Transformer that uses cross-attention to combine image features with the text prompt, not a text-only or image-only model.

Q60.

BERT is encoder-only and bidirectional — for classification, seeing the full sentence (both directions) at once typically gives richer context than GPT's causal left-to-right view, and BERT doesn't need generation capability, which classification doesn't require anyway.

Q61.

GPT is decoder-only and autoregressive, trained specifically to predict the next token — this is exactly what's needed to generate coherent free-form text one token at a time. BERT's masked-LM objective isn't designed for open-ended generation.

Q62.

Possible causes to investigate: a bug in the masking (e.g. padding not masked, so attention wastes weight on padding tokens), poor initialization, learning rate too high/low preventing proper training, or simply an undertrained model — visualize attention maps and check the masking logic first, since that's the most common bug.

SECTION F | LLM / GENERATIVE AI SCENARIOS

Q63.

Prefer RAG (Retrieval-Augmented Generation) over fine-tuning — internal docs change constantly, and RAG lets you update the knowledge base instantly without retraining, while also grounding answers in retrievable sources (reducing hallucination).

Q64.

Handle tone with prompt engineering/system prompts (cheap, fast to iterate), and handle product knowledge with RAG (also cheap to update). Full fine-tuning is the most expensive lever and should be reserved for cases where prompting and RAG genuinely aren't enough.

Q65.

Combine quantization, KV-cache optimization, request batching, and possibly a smaller/distilled model or MoE architecture — inference-time cost engineering, not just picking 'the best model' regardless of size.

Q66.

Prefer RAG grounded in the mission documents over fine-tuning, since fine-tuning needs a meaningful amount of domain data to avoid overfitting/forgetting general ability — RAG works even with zero labeled instruction data, just the raw documents.

Q67.

Ground responses in retrieved, verifiable source documents (RAG), add citation of sources, and add a verification/guardrail layer that flags low-confidence or unsupported claims before they reach the user.

Q68.

Options: extend context length via position-encoding tricks (RoPE scaling) if using a model that supports it, summarize/compress older turns into a condensed memory, or use retrieval over conversation history (treat past turns like a document store) instead of keeping everything in the raw prompt.

Q69.

Deploy a small, quantized/distilled LLM directly on the vehicle's onboard compute rather than relying on a cloud API — this trades some capability for guaranteed offline availability and low latency.

Q70.

Consider distillation to a smaller model, switching to an MoE architecture (only activates a subset of parameters per token), aggressive caching of common queries/responses, and quantization — usually a combination of these, chosen based on where the actual cost is concentrated.

Q71.

This requires careful alignment work (RLHF/DPO-style preference training) plus well-designed guardrails/classifiers that distinguish genuinely harmful requests from legitimate adjacent ones — a blanket keyword-block approach fails both by over-blocking legitimate use and under-blocking clever harmful phrasing.

Q72.

Consider RAG instead of fine-tuning when possible (data stays in a retrievable, access-controlled store rather than baked into model weights), and if fine-tuning is necessary, apply data anonymization/de-identification and techniques like differential privacy during training.

Q73.

Use RAG with explicit citation tracking — retrieve source passages, have the model reference them directly, and surface the citations alongside the generated answer so every claim is traceable back to a document.

Q74.

This requires a structured bias audit (systematic testing across demographic variations with controlled inputs), followed by mitigation — could include data rebalancing, fairness-aware fine-tuning/RLHF, or redesigning the prompt/pipeline to remove demographic signals entirely before scoring.

Q75.

Start with prompt engineering on an existing pretrained/API model — it's fast, cheap, and needs no training infrastructure. Escalate to RAG if the issue is missing knowledge, and only consider fine-tuning if neither prompting nor RAG solves the problem after real attempts.

Q76.

Use request batching, KV-cache reuse, and speculative decoding (a small draft model proposes tokens that the large model verifies in parallel) — these boost throughput without touching model weights or quality.

Q77.

Use chain-of-thought prompting (ask the model to reason step by step before answering) and/or tool use (let the model call a calculator/code execution tool for the actual arithmetic instead of trying to compute it purely via next-token prediction).

Q78.

This needs function calling / tool use (agentic design) — the LLM is given structured access to external APIs (calendar, email) and trained/prompted to invoke them with the right parameters, rather than just generating descriptive text.

Q79.

This is primarily a systems/architecture problem, not a modeling one — strict data isolation (separate model instances, isolated vector stores/RAG indices, no shared caches across tenants) must be enforced at the infrastructure level, since no amount of prompting can guarantee data never crosses tenant boundaries.

Q80.

Build an evaluation framework combining automated benchmarks relevant to the task, targeted test sets for known failure modes, and structured human evaluation (side-by-side comparisons with clear rubrics) — 'it feels better' anecdotally is not a launch decision process.

Q81.

Could be either: a knowledge gap (fix with domain fine-tuning or RAG over legal corpora) or a tokenization gap where rare legal terms get split into many inefficient sub-word tokens (fix by extending/adapting the tokenizer's vocabulary for the domain) — diagnose by checking how the terms actually get tokenized before choosing a fix.

Q82.

Use an existing pretrained model via API or open weights, and fine-tune (or just prompt/RAG) on top of it for the specific use case — pretraining a frontier-scale LLM from scratch is a multi-million-dollar undertaking that's almost never the right call for a startup.

SECTION G | SYSTEM DESIGN, MLOPS & PRODUCTION SCENARIOS

Q83.

Design explicit fallback behavior (e.g. hand control back to the driver, or a conservative safe-stop maneuver) triggered by confidence/uncertainty thresholds and sensor-disagreement checks — a safety-critical system must degrade gracefully, not just try to be as accurate as possible and hope.

Q84.

Implement regular checkpointing (periodically save model/optimizer state to disk) so training can resume from the last checkpoint instead of restarting from scratch after any failure — essential for any long-running distributed training job.

Q85.

This is data/concept drift — user behavior and content have shifted since training. Set up ongoing monitoring of input feature distributions and live performance metrics, with alerts and a retraining pipeline (scheduled or trigger-based) to keep the model current.

Q86.

This demands extremely thorough offline validation across a huge range of edge cases before launch, favoring simpler/more robust models over marginally more accurate but less predictable ones, and building in conservative fallback behavior — you're optimizing for provable robustness, not just benchmark accuracy, since there's no 'ship a fix later.'

Q87.

Use a canary/gradual rollout — release the new model to a small percentage of traffic first, monitor key metrics closely, and progressively increase exposure only if the new version proves stable/better; keep an instant rollback path to the old model.

Q88.

Fix random seeds across all libraries (numpy, framework, data shuffling), pin exact library/environment versions, and control any other sources of nondeterminism (e.g. GPU nondeterministic ops) — reproducibility is a basic engineering requirement, not an afterthought.

Q89.

Combine caching of frequent results, approximate nearest-neighbor search (rather than exact comparison against the full catalog), and possibly a simpler/faster model for the real-time path with a heavier model used offline to precompute candidates — strict latency at this scale forces engineering tradeoffs beyond just model accuracy.

Q90.

The choice should be driven by business cost: if missing fraud (false negative) is far costlier than a false alarm (false positive), prioritize recall (with acceptable precision); if false alarms are very costly (e.g. blocking legitimate customers), prioritize precision. AUC is useful for comparing models overall but doesn't by itself set the operating threshold — that decision needs explicit cost/business input.

Q91.

Extensive staged validation: offline testing on huge held-out datasets, simulation across rare/edge-case scenarios, then a staged rollout (small fleet first, monitored closely) before wide OTA release — real-world millions-of-cars deployment is the last, not the first, validation step.

Q92.

This points to a missing monitoring/validation gate in the pipeline — add automated checks (e.g. minimum accuracy threshold on a held-out set, sanity checks on output distributions) that must pass before a new model is allowed to go live, plus alerting so failures are caught immediately, not a week later.

Q93.

Use distributed training — data parallelism (split data across multiple machines/GPUs, each with a model copy, gradients synced) is the typical starting point; if the model itself is also too large for one device, add model parallelism (split the model across devices) as well.

Q94.

If the model itself doesn't fit on one GPU, you need model parallelism (split the model's layers/parameters across GPUs) regardless of batch size; data parallelism alone (same full model copied to each GPU) doesn't solve a model that's too big for one device. Large-scale training often combines both.

Q95.

This depends on utilization pattern: sporadic/bursty workloads favor cloud (pay only when used, elastic scaling); consistently high, predictable utilization over a long time horizon can make on-premise cheaper — the answer isn't universal, it depends on workload steadiness and time horizon, plus factors like data sensitivity/compliance.

Q96.

Deleting the raw data is straightforward, but if a model was trained on it, the model may have implicitly memorized that data — this raises the 'machine unlearning' problem, which may require retraining the model without that user's data (expensive) or specialized unlearning techniques, and needs a data-retention/architecture strategy that anticipates such requests in advance.

Q97.

This is train-serve skew — features computed differently (or from different data sources) at training time vs serving time. Fix by using a shared feature pipeline/feature store for both training and serving, and add monitoring that compares live input distributions against training-time distributions.

Q98.

Raw softmax outputs are notoriously overconfident and not reliable uncertainty estimates. Use proper uncertainty quantification: ensembling (train multiple models, look at prediction spread), Bayesian neural network approaches, or Monte Carlo dropout at inference — and calibrate the resulting probabilities before they inform mission-critical decisions.

Q99.

Structured approach: confirm it's overfitting (check the train/val curves over time), check for data leakage (is test data truly held out and non-overlapping with train?), check dataset size and label quality, then apply fixes in order of cost — more data/augmentation, regularization/dropout, reduce model complexity, and re-validate the train/test split methodology itself, since a 49-point gap this large sometimes signals a broken split rather than pure overfitting.

Q100.

Use a structured framework: (1) clarify the problem and business objective/constraints first — don't jump to models; (2) discuss data — what's available, how it's labeled, quality issues; (3) propose a model approach with reasoning (why this architecture, referencing constraints like latency/interpretability/data size); (4) define evaluation metrics tied to business cost; (5) cover deployment/serving and monitoring/retraining strategy. Interviewers are grading the structured thinking and the questions you ask, far more than which specific model name you land on.